

Explication du notebook de construction des portefeuilles factorielles long-short

📌 Bloc 1 : Import des bibliothèques Python de base

📄 Ce bloc charge les deux bibliothèques fondamentales nécessaires :

- Pandas pour la gestion des données tabulaires (DataFrame),
- Numpy pour les calculs numériques (moyennes, arrays, etc.).

📌 Bloc 2 : Définition des chemins d'accès aux fichiers

📄 Ce bloc définit les chemins d'accès :

- *input_file* : le fichier Excel contenant les rendements journaliers des actions classées par facteur (chaque feuille Excel = un facteur),
- *output_file* : l'emplacement où sera enregistré le fichier final contenant les rendements des portefeuilles factoriels.

📌 Bloc 3 : Chargement des rendements journaliers d'un facteur

📄 Ici, on commence par charger la feuille "Momentum(i) Rendement Journalie" du fichier Excel, qui contient les rendements des actions :

- Renomme la première colonne en "Date" si nécessaire,
- Convertit les dates au bon format,
- Fixe "Date" comme index de la table,
- Trie les dates par ordre chronologique.

📌 Bloc 4 — Chargement de toutes les feuilles Excel (facteurs)

📄 Ce bloc permet d'obtenir **la liste de tous les noms de feuilles** dans le fichier Excel. Chaque feuille correspond à un facteur différent (Momentum, ROA, etc.). Cette liste sera utilisée pour boucler automatiquement sur tous les facteurs du fichier.

```

## Import des bibliothèques
import pandas as pd
import numpy as np

7.0s

# === FICHIERS ===
input_file = "C:/Users/Lenovo i7/OneDrive - ENSEA/Stage CGF Gestion/Documents/Mémoire/Data/EDOUARD/Données Modele HMM-FSHMM(4).xlsx"
output_file = "C:/Users/Lenovo i7/OneDrive - ENSEA/Stage CGF Gestion/Documents/Mémoire/Data/EDOUARD/Rendement/rendements_portfeuille"

0.0s

# === CHARGEMENT DES RENDEMENTS JOURNALIERS DES ACTIFS ===
rendements = pd.read_excel(input_file, sheet_name="Momentum(i) Rendement Journalie")
rendements.rename(columns={rendements.columns[0]: "Date"}, inplace=True)
rendements["Date"] = pd.to_datetime(rendements["Date"])
rendements.set_index("Date", inplace=True)
rendements.sort_index(inplace=True)

2.5s

# === CHARGER TOUTES LES FEUILLES COMME FACTEURS (INCLUS momentum) ===
xls = pd.ExcelFile(input_file)
facteurs = xls.sheet_names # Inclut "Momentum(i) Rendement Journalie"

```

Bloc 5 : Calcul des rendements mensuels des portefeuilles long-short (approche vectorielle)

 Ce bloc permet de calculer, pour chaque facteur :

1. **Une fois par mois**, les pondérations d'un portefeuille long-short :
 - **Long** sur les 20 % d'actions les mieux classées selon le facteur,
 - **Short** sur les 20 % d'actions les moins bien classées.
2. On tient compte de la nature du facteur :
 - S'il faut le **minimiser** (ex. : volatilité, beta), on trie de manière croissante,
 - Sinon, on trie de manière décroissante.
3. Les pondérations sont appliquées aux **rendements journaliers** des actions sur le mois, pour obtenir le **rendement journalier du portefeuille factoriel**.
4. Résultat : pour chaque facteur, on a une série temporelle des rendements quotidiens d'un portefeuille long-short exposé uniquement à ce facteur.

```

# === STOCKAGE DES RÉSULTATS ===
portefeuille_rendements = {}

for facteur in facteurs:
    print(f"Traitement du facteur : {facteur}")

    df = pd.read_excel(input_file, sheet_name=facteur)
    df.rename(columns={df.columns[0]: "Date"}, inplace=True)
    df["Date"] = pd.to_datetime(df["Date"])
    df.set_index("Date", inplace=True)
    df.sort_index(inplace=True)

    rendement_portefeuille = pd.Series(index=rendements.index, dtype="float64")

    monthly_groups = df.groupby([df.index.year, df.index.month])

    for (year, month), group in monthly_groups:
        try:
            start_date = group.index.min()
            end_date = group.index.max()

            # Nettoyage et conversion en numérique
            factor_scores = df.loc[start_date].apply(pd.to_numeric, errors='coerce').dropna()
            factor_scores = factor_scores[factor_scores.index.isin(rendements.columns)]

            # Trier les actifs selon le facteur
            facteur_minimisation = ["volatilité(i) la volatilité".lower(), "risk(i) beta".lower()]
            if facteur.strip().lower() in facteur_minimisation:
                sorted_assets = factor_scores.sort_values(ascending=True) # On MINIMISE
            else:
                sorted_assets = factor_scores.sort_values(ascending=False) # On MAXIMISE

            n = len(sorted_assets)
            top_20 = sorted_assets.head(int(0.2 * n)).index
            bottom_20 = sorted_assets.tail(int(0.2 * n)).index

            # Poids des actifs
            weights = pd.Series(0, index=rendements.columns, dtype="float64")
            weights[top_20] = 1 / len(top_20)
            #weights[bottom_20] = -1 / len(bottom_20)

            # Rendements journaliers du portefeuille
            monthly_returns = rendements.loc[start_date:end_date].dot(weights)
            rendement_portefeuille.loc[start_date:end_date] = monthly_returns

        except Exception as e:
            print(f"▲ Erreur en {year}-{month} pour le facteur {facteur} : {e}")

```

📌 Bloc 6 — Réunion des rendements de tous les facteurs dans un tableau final

📄 Ce bloc permet de convertir le dictionnaire *portefeuille_rendements* (où chaque clé est un facteur et chaque valeur est une série temporelle de rendements journaliers) en un tableau final de type DataFrame.

Ce tableau contient donc :

- Les dates en lignes (index),
- Les facteurs en colonnes,
- Les rendements journaliers long-short en valeurs.

📌 Bloc 7 — Export final des résultats vers un fichier Excel

📄 Ce bloc exporte le tableau resultats vers un fichier .xlsx (chemin spécifié dans `output_file`).

```
# === RÉUNION DES RÉSULTATS DANS UN TABLEAU FINAL ===
resultats = pd.DataFrame(portefeuille_rendements)

# === EXPORT VERS EXCEL ===
resultats.to_excel(output_file)
print(f"\n✅ Fichier de résultats généré avec succès : {output_file}")
```